

CSE 1113

Computer Programming

Sumaya Kazary

Assistant Professor

Department of Computer Science and Engineering
Dhaka University of Engineering & Technology, Gazipur

Acknowledgement

Thanks to the authors of all the books and online tutorials used in this slide.

User Defined Function

Part - 3

- **Preview of function calls**
- **Functions And Arrays**
- **Recursion**
- **Use of return() and exit()**

Types of Function Calls

- **Call by value**
 - Copy of argument passed to function
 - Changes in function do not affect original
 - Use when function does not need to modify argument
 - Avoids accidental changes

TYPES OF FUNCTION CALLS

- **Call by Reference:**

- When a function is called by an argument/parameter which is a pointer (address of the argument) the copy of *the address of the argument* is passed to the function.
- Therefore, a possible change on the data at the referenced address change the original value of the argument.
 - Passes original argument
 - Changes in function effect original
 - Only used with trusted functions

Example : swap function

```
#include <stdio.h>

void swap(int a , int b)
{
    int temp;
    temp=a;
    a=b;
    b=temp;
}

int main()
{
    int x=3, y=2;
    printf("before: x=%d, y=%d\n",x,y);
    swap(x,y);
    printf("after : x=%d, y=%d\n",x,y);
}
```

```
#include <stdio.h>

void swap(int *a , int *b)
{
    int temp;
    temp=*a;
    *a=*b;
    *b=temp;
}

int main()
{
    int x=3, y=2;
    printf("before: x=%d, y=%d\n",x,y);
    swap(&x,&y);
    printf("after : x=%d, y=%d\n",x,y);
}
```

Recursion Introduction

- Most mathematical functions are described by a simple formula
 - $C = 5 \times (F - 32) / 9$
 - It is trivial to program the formula
- Mathematical functions are sometimes defined in a less standard form
 - $f(0) = 0$ and $f(x) = 2f(x-1) + x^2$
 - Then, $f(0) = 0$, $f(1) = 1$, $f(2) = 6$, $f(3) = 21$, ...
- A function that is defined in terms of itself is *recursive*, and C allows recursive functions

Example

- Factorial: $N! = N * (N-1)!$
- Fibonacci Number: $F(n) = F(n-1) + F(n-2)$
- $x^n = x \times x^{n-1}$
- Tower of Hanoi

Example: factorial

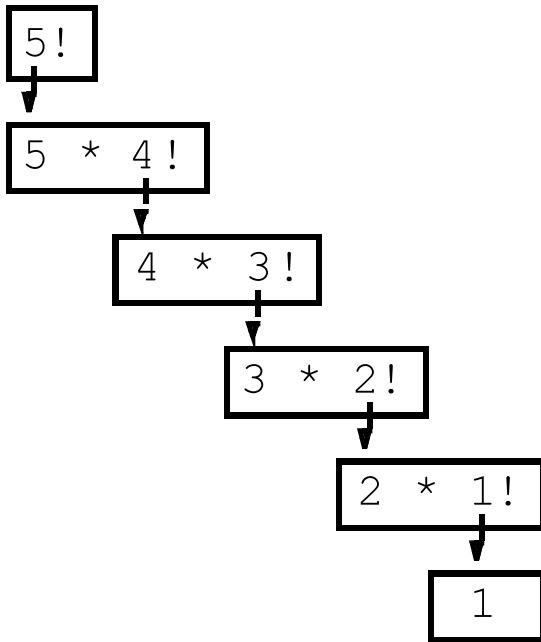
- Example: factorials
 - $5! = 5 * 4 * 3 * 2 * 1$
 - Notice that
 - $5! = 5 * 4!$
 - $4! = 4 * 3! \dots$
 - Can compute factorials recursively using factorials
 - Solve base case ($1! = 0! = 1$) then plug in
 - $2! = 2 * 1! = 2 * 1 = 2;$
 - $3! = 3 * 2! = 3 * 2 = 6;$

Factorial

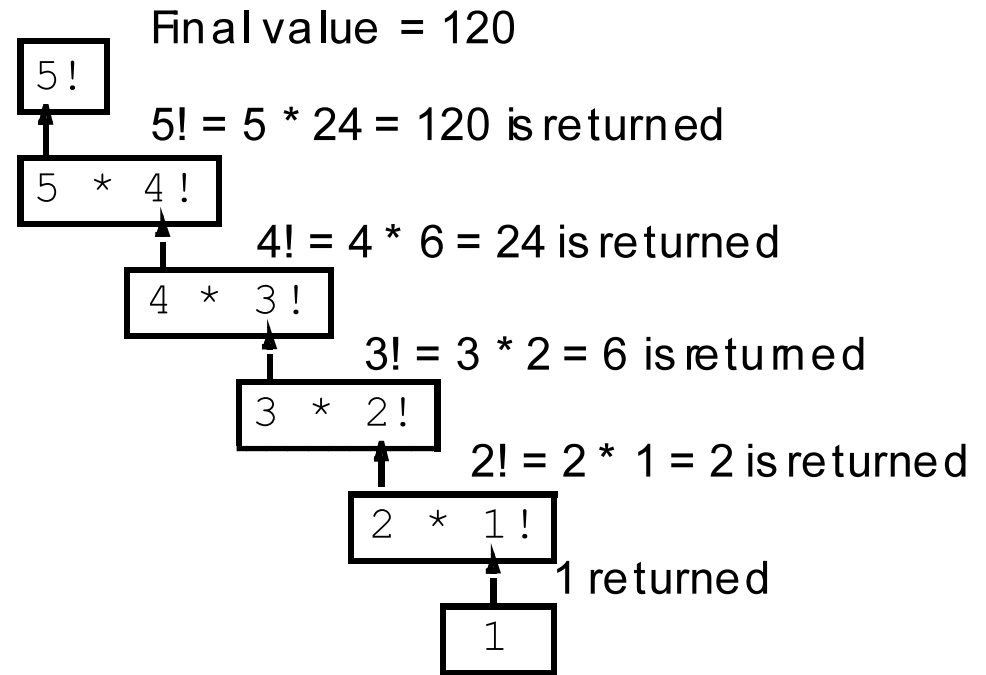
- The following function computes $n!$ recursively, using the formula $n! = n \times (n - 1)!$:

```
int factorial(int n)
{
    int t;
    if (n <= 1)
        t=1;
    else
        t=n * factorial(n-1);
    return t;
}
```

Recursion



(a) Sequence of recursive calls



(b) Values returned from each recursive call.

Recursion

- A function that calls itself is known as *recursive function* and this technique is known as **recursion** in C programming.
- **Fundamental Rules of Recursion**
 - Base Case: Must have some base cases which can be solved without recursion
 - Making Progress: for the case that are to be solved recursively, the recursive call always make progress toward the base case
- **If you do not follow the above rule, your program would not terminate**

Example of recursion in C programming

- **Write a C program to find sum of first n natural numbers using recursion.** [Note: Positive integers are known as natural number i.e. 1, 2, 3....n]

```
#include <stdio.h>
```

```
int add(int n);
```

```
int main(){
```

```
    int num, sum;
```

```
    printf("Enter a positive integer:\n");
```

```
    scanf("%d",&num);
```

```
    sum=add(num);
```

```
    printf("\n sum=%d", add);
```

```
}
```

```
int add(int n){
```

```
    if(n==0) return n;
```

```
    else return n+sum(n-1);
```

```
}
```

Output

Enter a positive integer: 5

Sum: 15

How the sum(5) executed?

sum(5)
=5+sum(4)
=5+4+sum(3)
=5+4+3+sum(2)
=5+4+3+2+sum(1)
=5+4+3+2+1+sum(0)
=5+4+3+2+1+0
=5+4+3+2+1
=5+4+3+3
=5+4+6
=5+10
=15

- Every recursive function must be provided with a way to end the recursion.
- In this example when, n is equal to 0, there is no recursive call and recursion ends.

Advantages and Disadvantages of Recursion

❑ **Advantages:**

- Recursion is more elegant and requires few variables which make program clean.
- Recursion can be used to replace complex nesting code by dividing the problem into same problem of its sub-type.

❑ **Disadvantages:**

- In other hand, it is hard to think the logic of a recursive function. It is also difficult to debug the code containing recursion.

Example #2

```
/* Source Code to calculate power using recursive function */
int power(int n1,int n2);
int main() {
    int base, exp;
    printf("Enter base number: ");
    scanf("%d",&base);
    printf("Enter power number(positive integer): ");
    scanf("%d",&exp);
    printf("%d^%d = %d", base, exp,power(base, exp));
    return 0;
}
int power(int base,int exp) {
    if (exp){
        return (base * power(base, exp-1));
    }
    return 1;
}
```

Example #2 : Output

Enter base number: 3

Enter power number(positive integer): 3

$3^3 = 27$

Arrays & Function

- In C programming, a single array element or an entire array can be passed to a function.
- Both one-dimensional and multi-dimensional array can be passed to function as argument.

Passing One-dimensional Array In Function

- ❑ A single element of an array to function

```
void display(int a){  
    printf("%d",a);  
}
```

```
int main(){  
    int c[]={2,3,4};  
    display(c[2]);  
    return 0;  
}
```

output :

4

- ❑ Passing entire one-dimensional array to a function

```
float average(float a[]);  
int main(){  
    float avg,c[]={23.4,55,22.6,3,40.5};  
    avg=average(c); //Only name of array is passed as argument.  
    printf("Average age=%.2f",avg);  
    return 0;  
}  
float average(float a[]){  
    int i;  
    float avg, sum=0.0;  
    for(i=0;i<5;++i){  
        sum+=a[i];  
    }  
    avg =(sum/5);  
    return avg;  
}
```

Passing Multi-dimensional Array In Function

```
void Function(int c[2][2]);
int main(){
    int c[2][2],i,j;
    printf("Enter 4 numbers:\n");
    for(i=0;i<2;++i)
        for(j=0;j<2;++j){
            scanf("%d",&c[i][j]);
        }
    Function(c); /* passing multi-dimensional array to function */
    return 0; }
void Function(int c[2][2]){//void Function(int c[][2])is
    also valid
    int i,j;
    printf("Displaying:\n");
    for(i=0;i<2;++i)
        for(j=0;j<2;++j)
            printf("%d\n",c[i][j]);
}
```

Questions?